

Technical Proposal: Optimizing B2B Recommendation Architectures via Graph Machine Learning Principles

Our current B2B recommendation engine uses collaborative, content based, and hybrid algorithms to match Retailers with Distributors.

1. Identified Engineering Vulnerabilities & Proposed Fixes :

Vulnerability 1: The $O(N)$ Database I/O Bottleneck ($N+1$ Query Pattern)

- **Location:** collaborative.py (user_based_cf / matrix_factorization_like_cf)
- **The Issue:** The algorithm loops through similar users (for uid in co_users) and makes an independent Django ORM database request during every single iteration. As our user base scales across Nepal, this will saturate database connection pools and cause severe API timeouts.
- **Proposed Solution:** Refactor the loop to fetch all co user interaction metrics in one bulk database query using .values(). We will map these connections in memory using a Python default dict cache, reducing network overhead to a flat $O(1)$ database execution block.

Vulnerability 2: Ranking Order Scrambling via standard SQL IN Operators

Location: hybrid.py (get_hybrid_recommendations)

The Issue: The engine properly calculates and sorts candidate scores in Python memory. However, passing these sorted entity IDs directly into a standard SQL .filter(id__in=[...]) query causes the database engine to return rows according to its default index sequence or physical page storage layout. Our hyper tuned mathematical rankings are completely discarded before hitting the UI.

Proposed Solution: Update the retrieval step to use Django's Case and When conditional expressions. This forces the underlying SQL database to return records in the exact calculated sequence determined by our hybrid engine.

Vulnerability 3: The Hard Cold Start Trap

Location: Global Execution Workflow

The Issue: If a newly registered business has zero interaction logs (BusinessInteraction), or if a latent relationship requires inferring patterns across multiple hops (e.g., Store A -> Imports Steel -> Matches Store C's demand), the collaborative modules instantly trigger empty terminations (return []). New users receive a blank screen.

Proposed Solution: Introduce an embedding driven structural retrieval layer that evaluates cosine similarity over shared structural attributes rather than strict transaction history.

1. Academic Alignment: Translating GFT & OFA into Our System

By evaluating our architecture against the GFT (NeurIPS 2024) and OFA (ICLR) papers, we can automate our matchmaking pipeline to be parameter driven, searchable, and highly interactive without manual human data tagging :

[https://openreview.net/pdf?id=Kdcqzfypry#:~:text=In%20this%20work%2C%20we%20investigate%20a%20unified,new%20graph%20domains%3B%20iv\)%20Scaling%20Law%20Emergence.](https://openreview.net/pdf?id=Kdcqzfypry#:~:text=In%20this%20work%2C%20we%20investigate%20a%20unified,new%20graph%20domains%3B%20iv)%20Scaling%20Law%20Emergence.)

<https://www.tigergraph.com/glossary/graph-machine-learning/>

https://proceedings.iclr.cc/paper_files/paper/2024/file/57faf5642eb06e0602b95f6aa989b38a-Paper-Conference.pdf

Computation Tree Representation: Instead of treating our database as disconnected tables, we will model the platform as a (Retailer-> Category -> Product -> Distributor). Matching will evaluate local structural paths (neighborhood subgraphs), allowing the engine to discover non obvious supplier connections.

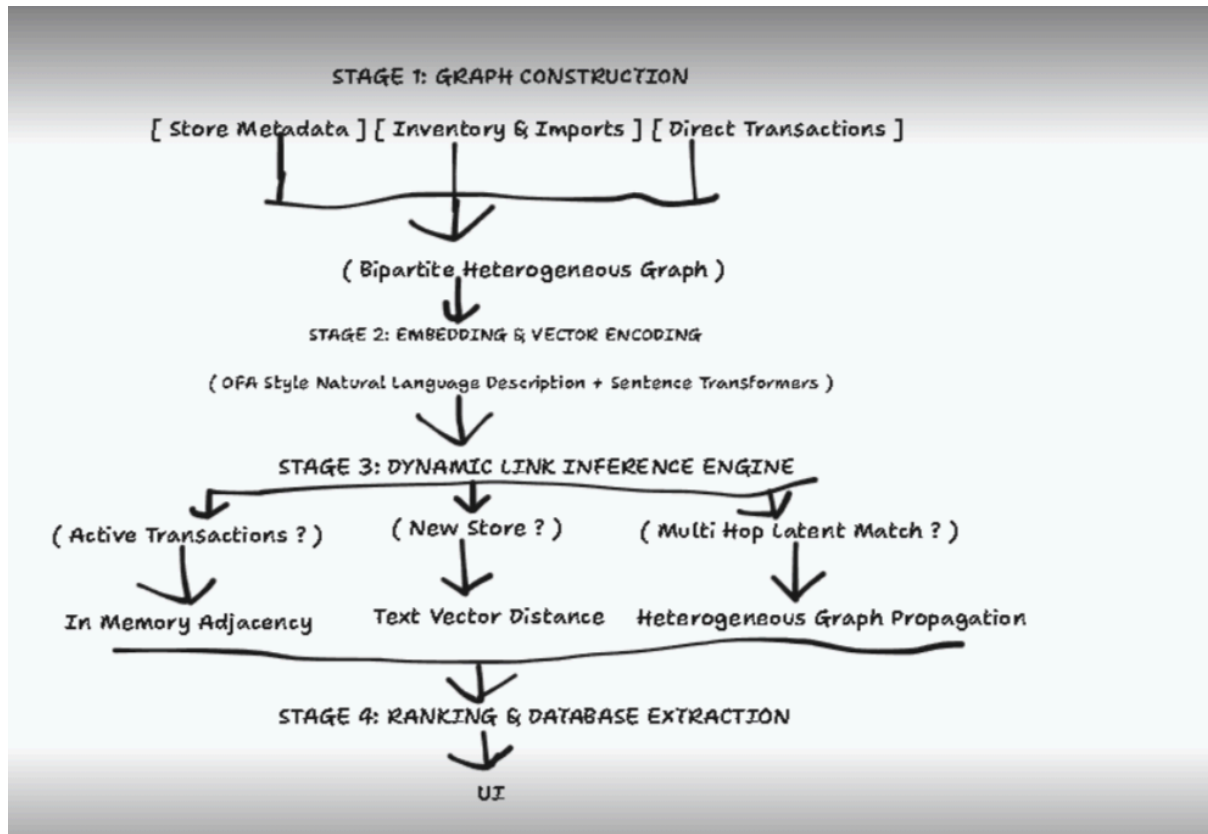
Continuous Feature Spaces (Replacing Bounding Boxes): The current script uses rigid, manual Min/Max pricing boundaries that break easily when high variance inventories are added. Following GFT's tokenization logic, we propose mapping pricing vectors and textual category definitions into dense, continuous embedding spaces. This enables high accuracy matching across highly diverse industries.

Pre Implementation Workflow:

1. Architectural Workflow Overview

To replace static, single hop database lookups with Latent Link Prediction (e.g automatically matching Store A which imports steel with Store C which sells steel), the platform is structured into a 4 Stage Automated Pipeline:

https://www.tldraw.com/f/r5PAoGqu_eZlnJVjhpGLs?d=v71.-330.1911.1044.page



2. Stage by Stage Implementation Details :

Stage 1: Heterogeneous Graph Schema Definition

Stage 1: Heterogeneous Graph Schema Definition Instead of viewing the database as disconnected tables, we construct an in memory Heterogeneous Bipartite Graph(G) = (V, E, T_v, T_e)

Node Types (T_v):

V_{Store} : Retailers, Distributors, Wholesalers.

$V_{Product}$: Specific SKU items.

$V_{Category}$: Product categories (e.g: Ceramics, Hardware, Steel).

$V_{\text{Attribute}}$: Operations (e.g: *Imports, Exports, Manufactures*).

Edge Types (T_e):

$E_{\text{Transacted}} : V_{\text{Store}} \text{ Buys from } V_{\text{Store}}$ (Explicit transaction).

$E_{\text{Offers}} : V_{\text{Store}} \text{ Supplies } V_{\text{Product}}$ (Inventory relation). E_{Action} :

$E_{\text{Categorized}} : V_{\text{Product}} \text{ Belongs to } V_{\text{Category}}$

$E_{\text{Action}} : V_{\text{store}} \text{ Import } V_{\text{Category}}$

How this resolves the Store A / C Latent Link:

If Store A sells Ceramic, but adds an edge $E_{\text{Action}}(\text{Imports}) \rightarrow V_{\text{Category}}(\text{Steel})$, and Store C is connected via $E_{\text{Categorized}} \rightarrow V_{\text{Category}}(\text{Steel})$, a 2 hop structural path $P = (\text{Store}_A \rightarrow \text{Steel} \rightarrow \text{Store}_C)$ is formed immediately without requiring past sales history between them.

Stage 2: Feature Embedding Pipeline (OFA Alignment) : To convert unstructured text, pricing arrays, and geographic parameters into a unified mathematical space, every node is transformed into a continuous dense feature vector .

Algorithmic Step 2.1: Textual Description Synthesis

For every store entity, a standardized textual representation S_v is generated:

$S_v = \text{Concat}(\text{Name, Role, Location, Categories, Import Features})$

Algorithmic Step 2.2: Continuous Vector Encoding:

Pass S_v through a pre trained Transformer Encoder.

Algorithmic Step 2.3: Hybrid Feature Vector Fusion

Combine textual semantic embeddings with normalized geographic coordinates and median pricing tiers.

Stage 3: Algorithmic Formulations for Recommendation Engines

The inference engine executes three complementary scoring algorithms in parallel, dynamically blending their outputs based on user profile state.

Algorithm A: Optimized In Memory Jaccard Similarity (For Active Users):

For target user u , extract its interaction neighborhood $N(u)$. Calculate candidate overlap across co users in memory:

$$\text{Sim}_{\text{jaccard}}(u, v) = |N(u) \cap N(v)| / |N(u) \cup N(v)|$$

Algorithm B: Continuous Cosine Similarity (Cold Start) :

If $|N(u)| = 0$, compute dense vector similarity between the target store u and candidate store v :

$$\text{Sim}_{\text{Cosine}}(u, v) = h_u \cdot h_v / \|h_u\| \|h_v\|$$

Algorithm C: Multi Hop Structural Graph Inference (Latent Link Prediction)

To compute proximity across multi-hop paths (e.g. $\text{Store}_A \rightarrow \text{Steel} \rightarrow \text{Store}_C$), we apply a Random Walk with Restart (RWR) or Message Passing Propagation step:

1. Initialize state probability vector p_0 at target node u (where $p_0(u) = 1$, and 0 elsewhere).
2. Iteratively propagate probability across transition matrix M with restart probability $(0, 1)$
3. The final stationary distribution vector p^* yields the latent structural proximity score for all stores across the entire supply network, regardless of direct sales history.

Stage 4: Dynamic Hybrid Scoring & Database Preservation

Step 4.1: Weighted Score Blending

The final score $S(u, v)$ for candidate v relative to target user u is calculated via configurable weight parameters:

$$S(u, v) = w_1 \cdot \text{Sim}_{\text{jaccard}}(u, v) + w_2 \cdot \text{Sim}_{\text{Cosine}}(u, v) + w_3 \cdot p_v^*$$

Step 4.2: Enforcing Ordered Retrieval via Conditional SQL

To ensure that the mathematical sequence determined by $S(u, v)$ is preserved when retrieving database records, the output IDs are mapped using explicit conditional ordering:

```
ORDER BY CASE id WHEN id1 THEN 1 WHEN id2 THEN 2 ... END
```

3. Implementation Workflow & Data Flow Steps :

[Step 1: Data Trigger]

-> Vendor updates profile or adds inventory (e.g: Store A adds Imports Steel).

[Step 2: Asynchronous Graph Index Update]

-> Background worker updates the local Adjacency Matrix and regenerates vector.

[Step 3: Candidate Generation]

-> Engine pulls top K candidate nodes via combined Vector Distance + Adjacency paths.

[Step 4: Hybrid Scoring]

-> Evaluates Algorithm A, B, or C depending on user historical density.

[Step 5: Database Payload Response]

-> Returns strictly ordered vendor profiles via Django Case, when queries to the front end.

Lightweight Graph Transformer Pipeline

As discussed, our goal is to build a structured, end to end Graph Transformer pipeline that is highly lightweight. We are prioritizing system architecture and pipeline flow over maximum accuracy for this initial build.

1. Research: Which Graph Transformer to Use?

To keep the PC load minimal while still using state of the art graph attention, we will avoid massive LLMs and custom AFormer architectures. Instead, we will combine two highly optimized, lightweight models:

For Text Features (Node Embeddings): It is a tiny SentenceTransformer model created by Microsoft. It is exceptionally fast, designed specifically to run on standard CPUs, and only takes up 80MB of disk space.

For the Graph Transformer (Structural Inference): PyTorch Geometric's , Instead of building a complex Graph Transformer from scratch, PyTorch Geometric (PyG) has a built in TransformerConv layer. It applies standard multi head attention to graph neighbors (exactly like the paper suggests) but is highly optimized for standard RAM and CPUs.

2. The Ranking Algorithm (How we sort the recommendations)

Once the Graph Transformer processes the network, every store (node) is represented as a dense mathematical vector. To rank the recommendations (e.g: matching Store C to Store A), we will use Cosine Similarity + Top K Sorting.

The ranking algorithm works as follows:

1. Extract the final vector h_u for the target retailer.
2. Extract the vectors h_v for all potential distributors in the graph.
3. Compute the Cosine Similarity score for every pair:

$$\text{Score}(u, v) = h_u \cdot h_v / \|h_u\| \|h_v\|$$

4. Sort: Order the results descending from 1.0 (perfect match) to -1.0 (complete mismatch).
5. Top K: Return the top 10 highest scoring IDs to the frontend.

3. Hardware Requirements & PC Load (Local Prototyping)

Because we are using MiniLM and a shallow PyG TransformerConv network, the system requirements are extremely low. You can run this comfortably on a standard developer laptop without a dedicated GPU.

GPU Requirement: 0 GB (Can run 100% on CPU for this).

RAM Load: 1.5 GB to 3 GB maximum during the graph training loop (depending on the mock dataset size).

Storage/Disk: < 200 MB for downloading the open source model weights.

CPU Load: Will utilize 30-50% of a standard Intel i5/Ryzen 5 processor during the embedding phase, and drop to near 0% during standard API inference requests.

4. Structured Implementation Pipeline (Workflow)

We will build the pipeline in four sequential steps.

Phase 1: Data Preparation & Graph Construction

- Extract mock database tables (Retailers, Distributors, Products).
- Combine store metadata (location, categories, business type) into a single text string per store.
- Build a NetworkX/PyG graph where edges represent physical connections (e.g: Store A -> Imports -> Steel).

Phase 2: Lightweight Node Embedding

Pass the store text strings through the all MiniLM L6 v2 transformer.

This generates a lightweight 384 dimensional feature vector for every node in the graph, resolving the cold start problem.

Phase 3: Graph Transformer Message Passing

- Feed the graph structure and the MiniLM vectors into a 2 layer TransformerConv network.
- The transformer allows nodes to pay attention to their neighbors (e.g: Store C pays attention to the Steel category node), updating their vectors to include supply chain context.

Phase 4: Ranking & API Delivery

- Expose a FastAPI endpoint
- The API runs the Cosine Similarity ranking algorithm, sorts the matches, and returns a JSON list of recommended Store IDs.